

PAYMENT ENTITY – SQL DATABASE REPORT

1. Objective

To create and manage a Payment Entity using SQL by implementing table creation, constraints, payment records, and queries for successful, failed and pending transactions.

2. Payment Entity

The Payment Entity is used to store and manage payment information associated with customer orders.

Attributes

Attribute	Description
Payment_ID	Unique identifier for each payment
Order_ID	Related order identification
Payment_Date	Date of payment
Payment_Mode	UPI, Card, Cash, etc.
Payment_Status	Success, Failed or Pending
Transaction_Amount	Amount paid

3. Create Payment Table

The Payment table is created to store payment-related information.

```
SQL> CREATE TABLE Payment(PAYMENT_ID VARCHAR2(10),ORDER_ID VARCHAR2(7),PAYMENT_DATE DATE,PAYMENT_MODE VARCHAR2(20),PAYMENT_STATUS VARCHAR2(20),TRANSACTION_AMOUNT NUMBER(10,2));  
Table created.
```

4. Constraints

Constraints are applied to maintain data integrity and establish the relationship between the Payment and Orders tables.

Primary Key

```
SQL> ALTER TABLE Payment ADD CONSTRAINT PK_PAYMENT PRIMARY KEY(PAYMENT_ID);  
Table altered.
```

Foreign Key

```
SQL> ALTER TABLE Payment ADD CONSTRAINT FK_PAYMENT_ORDER FOREIGN KEY(ORDER_ID) REFERENCES Orders(ORDER_ID);  
Table altered.
```

NOT NULL Constraint

```
SQL> ALTER TABLE Payment MODIFY(PAYMENT_MODE NOT NULL,PAYMENT_STATUS NOT NULL);  
Table altered.
```

Check Constraint

```
SQL> ALTER TABLE Payment ADD CONSTRAINT CHK_PAYMENT_AMOUNT CHECK(TRANSACTION_AMOUNT>0);  
Table altered.
```

5. Insert Payment Records

Payment records are inserted into the Payment table with different payment modes, dates, statuses and transaction amounts.

```
SQL> INSERT INTO Payment VALUES ('P001','ORD0001',DATE '2026-09-01','UPI','Success',1500);  
1 row created.  
SQL> INSERT INTO Payment VALUES ('P002','ORD0002',DATE '2026-09-02','Card','Success',2500);  
1 row created.
```

```
SQL> INSERT INTO Payment VALUES ('P003','ORD0002',DATE '2026-09-03','UPI','Pending',1800);  
1 row created.
```

```
SQL> INSERT INTO Payment VALUES ('P004','ORD0002',DATE '2026-09-04','Cash','Success',1200);  
1 row created.
```

```
SQL> INSERT INTO Payment VALUES ('P005','ORD0001',DATE '2026-09-05','UPI','Success',900);  
1 row created.
```

6. Display All Successful Payments

This query displays all payment transactions whose status is **Success**.

```

connected.
SQL> SELECT * FROM Payment WHERE PAYMENT_STATUS = 'Success';

PAYMENT_ID ORDER_I PAYMENT_DATE      PAYMENT_MODE      PAYMENT_STATUS
-----
--
TRANSACTION_AMOUNT
-----
P001        ORD0001 01-SEP-26          UPI              Success
              1500
P002        ORD0002 02-SEP-26          Card             Success
              2500
P004        ORD0002 04-SEP-26          Cash             Success
              1200

PAYMENT_ID ORDER_I PAYMENT_DATE      PAYMENT_MODE      PAYMENT_STATUS
-----
--
TRANSACTION_AMOUNT
-----
P005        ORD0001 05-SEP-26          UPI              Success
              900

```

→ Successful Payments result

7. Find All Failed Transactions

This query displays all payment transactions whose status is **Failed**.

```

SQL> SELECT * FROM Payment WHERE PAYMENT_STATUS = 'Failed';

no rows selected

```

→ Failed Payments result

8. Count Total Successful and Failed Payments

This query counts the total number of successful and failed payments.

```

SQL> SELECT PAYMENT_STATUS, COUNT(*) AS TOTAL_PAYMENTS FROM Payment WHERE PAYMENT_STATUS IN ('Success','Failed') GROUP BY PAYMENT_STATUS;

PAYMENT_STATUS      TOTAL_PAYMENTS
-----
Success              4

```

→ Count result

9. Update Failed Payments After Retry

This query updates failed payments to **Success** after a successful retry.

```
SQL> UPDATE Payment SET PAYMENT_STATUS = 'Success' WHERE PAYMENT_STATUS = 'Failed';
0 rows updated.
```

→ UPDATE result

10. Identify Pending Transactions

This query displays all payment transactions whose status is **Pending**.

```
SQL> SELECT * FROM Payment WHERE PAYMENT_STATUS = 'Pending';
```

PAYMENT_ID	ORDER_I	PAYMENT_DATE	PAYMENT_MODE	PAYMENT_STATUS

TRANSACTION_AMOUNT				

P003	ORD0002	03-SEP-26	UPI	Pending
	1800			

→ Pending Payments result

11. Conclusion

The Payment Entity was successfully created and implemented using SQL. Primary key, foreign key, NOT NULL and check constraints were applied to maintain data integrity. Payment records were inserted and analyzed using SQL queries. Successful, failed and pending transactions were identified and managed according to their status.